

Звіт до програмного проекту №4

студентки групи К-27

Макарової Софії

Варіант №19:

$m = 2$ поля

Поле 0: read – 5%, write – 40%

Поле 1: read – 5%, write – 5%

string – 45%

Схема захисту даних

Захист реалізовано за принципом *Fine-Grained Locking* з масивом із $m = 2$ м'ютексів, кожен з яких відповідає окремому полю. Операції *read* і *write* блокують лише м'ютекс свого поля через *std::lock_guard*, що забезпечує паралельність доступу до різних полів.

Оператор *string()* формує узгоджений snapshot, тому послідовно блокує обидва м'ютекси у фіксованому порядку ($0 \rightarrow 1$). Це усуває ризик дедлоків.

Схема є оптимальною: звичайні операції блокуються мінімально, а повне блокування під час *string* (45% операцій) виконується безпечно та контрольовано.

Умови / К-сть потоків

Умови / Потоки	1 (ms)	2 (ms)	3 (ms)
GoodExpCond	1605.1	3382.42	5995.93
EqualExpCond	1354.85	2948.81	5094.49
BadExpCond	1605.12	4494.05	7500.49

Висновки

Результати є очікуваними: операція *string*, яка блокує всю структуру, найбільше впливає на загальний час виконання. Тому у “поганому” профілі при 2–3 потоках час різко зростає через постійне очікування м'ютексів. “Рівномірний” профіль працює найшвидше завдяки меншій частці повних блокувань, а GoodExpCond демонструє середні показники. У всіх випадках збільшення кількості потоків закономірно збільшує час виконання через конкуренцію за ресурси.

Зроблено самостійно

- розроблена потокобезпечна структура даних *DataStructure*;
- реалізовано роботу з окремими mutex для кожного поля;
- створено генератор трьох типів тестових файлів (good/equal/bad);
- адаптовано good-профіль під відсотки *варіанту №19*;
- реалізовано систему запуску тестів для 1–3 потоків з окремими файлами на потік;
- виконано збір замірів і побудовано підсумкову таблицю;
- сформовано висновки та оформлено звіт.